

Relatório de Decisões de Implementação - Etapa 2

Projeto: Sistema de Gestão Hospitalar Dra. Yuska Maritan Brito

Autores: Gerson Gonçalves de Freitas, Nicolas Kleiton da Silva Melo, Rodrigo Monteiro Fortes de Oliveira, Tiago Varelo da Silva

1. Introdução

Na Etapa 2 do projeto, o objetivo principal foi migrar as lógicas de banco e regras de negócio para estruturas mais robustas dentro do próprio PostgreSQL (Stored Procedures, Triggers e Views) e integrar o banco de dados a uma aplicação final que consome esses dados utilizando o paradigma de Orientação a Objetos por meio de uma ORM. Este relatório detalha as motivações por trás das decisões arquiteturais tomadas durante essa evolução.

2. Decisões de Implementação: Triggers vs. Stored Procedures

Ao transferir a lógica para o banco de dados, a escolha entre usar um *Gatilho* (Trigger) ou um *Procedimento Armazenado* (Stored Procedure) seguiu a divisão entre **operações reativas/passivas** e **operações ativas/intencionais**.

Stored Procedures (Ações Ativas da Aplicação)

As Procedures foram escolhidas para fluxos operacionais completos em que a aplicação envia um comando intencional que envolve múltiplas etapas ou que dita o controle de uma transação.

- **sp_registrar_atendimento_completo**: Como a gravação de um atendimento exige a persistência atômica tanto do "cabeçalho" (Atendimento) quanto dos seus "itens" (Procedimentos Realizados), escolhemos agrupá-los em uma transação única via Procedure recebendo um payload **JSONB**. Isso impede que falhas locais (ex: um erro de inserção em um procedimento específico) deixem dados órfãos, graças ao bloco **EXCEPTION** do PL/pgSQL que garante o *rollback* da subtransação se qualquer etapa do fluxo quebrar.
- **sp_reajustar_escala**: Escolhida pois o usuário dispara ativamente uma atualização em massa (mudar de um turno para o outro). A verificação prévia de conflito foi muito mais fácil de isolar e escalar nessa rotina ativa.

Triggers (Reações Automáticas e Integridade)

As Triggers foram empregadas estritamente para proteção da integridade dos dados e operações invisíveis à aplicação, independentemente de quem emita o SQL (a ORM, o DBA via terminal, etc).

- **trg_check_sobreposicao_escala**: Escolhemos uma Trigger **BEFORE INSERT/UPDATE** em vez de deixar essa regra para a aplicação, pois as chaves **UNIQUE** nativas só cobrem validações na *mesma linha*. A trigger impede, na raiz do banco, que um residente burle a grade de horário ao ser alocado em duas unidades ao mesmo tempo.
- **trg_audita_atendimento** e **trg_atualiza_media_procedimentos**: Rotinas **AFTER** ideais para a criação de rastreabilidade (snapshots **JSONB**) e cálculo de campos derivados estatísticos. Fazer isso via

aplicação geraria tráfego de rede desnecessário, além de quebrar o encapsulamento do banco (qualquer alteração forçada via banco burlaria a auditoria, mas com Trigger, o histórico é inevitável).

3. Escolha da ORM

A ORM escolhida para consumir o PostgreSQL foi o **SQLAlchemy** (Python).

Justificativas para a escolha:

1. **Ecosistema:** Sendo a principal ferramenta de persistência em Python, o SQLAlchemy encaixou perfeitamente com a escolha do **Streamlit** (também em Python) para construir a interface visual do hospital, reduzindo o *overhead* tecnológico ao unificar backend e frontend em uma única stack.
2. **Abstração Relacional Poderosa:** Permitiu fazer o mapeamento objeto-relacional através de **DeclarativeBase** e da ferramenta **relationship()**, com navegação direta entre pai (Pessoa) e filhas (Residente/Paciente/Profissional) de forma muito elegante e nativa, inclusive com configuração flexível sobre a cardinalidade (Lazy/Eager loading, **uselist=False**, exclusões em cascata).
3. **Mapeamento de Views:** A ORM permitiu que tratássemos as views (**vw_pacientes_internados**, **vw_estatisticas_atendimentos_mensal**) como classes comuns que herdavam de **Base**, injetando chaves primárias compostas lógicas para que o SQLAlchemy consiga mapeá-las facilmente sem perder as tipagens.

4. Tratamento de Concorrência

Para atender o requisito de concorrência concorrente escalando residentes na grade, implementamos a estratégia de **Lock Otimista**.

- **Implementação:** Foi adicionada a coluna de rastreio de versão **version_id** na tabela **ESCALA**, configurando no modelo a propriedade **__mapper_args__ = {"version_id_col": version_id}** do SQLAlchemy.
- **Motivo de usar Otimista vs. Pessimista:** Num sistema de agendamento e alocação hospitalar, o bloqueio pessimista (**SELECT FOR UPDATE**) travaria as leituras das linhas. Vários gestores podem analisar a escala simultaneamente e a contenção seria enorme. Com o Lock Otimista, a fluidez de leitura de toda a grade é priorizada e garantida. O banco permite as duas transações em andamento e, caso as duas submetam uma atualização ao mesmo instante para a mesma unidade, o SQLAlchemy intercepta a defasagem do *version id*, cometendo a primeira e bloqueando a segunda de forma segura com um erro rastreável (**StaleDataError**), forçando apenas uma re-leitura amigável em caso de conflito, sem *deadlocks* do SGBD.